STM32Cube GitHub Preprocessing Pipeline

Technical Documentation

Enterprise Reference Document

Version 2.0 — June 2026

Abir Gharbi
STMicroelectronics — AI & Digital Tools Division
abir.gharbi@st.com

Contents

1 Executive Summary **2**
- 1.1 Project Overview . . . 2
- 1.2 Key Achievements . . . 2
- 1.3 System Context . . . 3

2 Architecture **4**
- 2.1 Pipeline Stages . . . 4
- 2.2 Repository Structure . . . 5
- 2.3 Configuration System . . . 5

3 Pipeline Stage Details **7**
- 3.1 Stage 1: Ingestion . . . 7
  - 3.1.1 Scripts . . . 7
  - 3.1.2 Output . . . 8
- 3.2 Stage 2: Cleaning . . . 8
- 3.3 Stage 3: Enrichment V2 . . . 8
  - 3.3.1 Issues Enrichment (`issues_to_docs_v2.py`) . . . 8
  - 3.3.2 Files Enrichment (`files_to_docs_v2.py`) . . . 9
- 3.4 Stage 4: Similarity . . . 9
- 3.5 Stage 5: Chunking V2 . . . 9
- 3.6 Stage 6: Evaluation . . . 9
- 3.7 Stage 7: Delivery . . . 10
  - 3.7.1 Export Scripts . . . 11
  - 3.7.2 Delivery Path Routing . . . 11

4 KB Upload & Deployment **12**
- 4.1 ST AI Bridge API . . . 12
- 4.2 Datasource Registry . . . 12
- 4.3 Upload Parameters . . . 13
- 4.4 Large File Handling . . . 13

5 Alfred Vision AI Integration **14**
- 5.1 Purpose . . . 14
- 5.2 Integration . . . 14

6 Persona Configuration **16**
- 6.1 Recommended Settings . . . 16
- 6.2 System Prompt . . . 16

7 Automation & Operations **17**
7.1 Full Workflow Execution . . . 17
7.2 Multi-Series Automation . . . 18
7.3 Upload Scripts . . . 18
7.4 Fully Automated KB Update (End-to-End) . . . 19
7.4.1 What the Script Does . . . 19
7.4.2 Scheduling for Always-Fresh KB . . . 19
7.4.3 GitHub Actions Cloud Automation (Recommended) . . . 20
7.4.4 Recommended Frequency . . . 20
7.4.5 Safety Guarantees . . . 20

8 Quality Assurance **21**
8.1 Test Suite . . . 21
8.2 Schema Validation . . . 22

9 ST ChatGPT Platform — Usage Guide **23**
9.1 Platform Overview . . . 23
9.2 How the Workflow Operates End-to-End . . . 23
9.3 Knowledge Base Content Structure . . . 24
9.3.1 File Organization in KB . . . 24
9.3.2 What Each Datasource Enables . . . 24

10 Persona Configuration — Best Practices **26**
10.1 Recommended Persona Settings . . . 26
10.2 System Prompt Design . . . 26
10.3 What Works Well (Proven Patterns) . . . 27
10.4 Known Limitations . . . 27
10.5 Anti-Patterns to Avoid . . . 28

11 KB Maintenance & Update Workflow **29**
11.1 When to Re-run the Pipeline . . . 29
11.2 Update Procedure . . . 29
11.3 Datasource Versioning Strategy . . . 29
11.4 Adding a New MCU Series . . . 30

12 Evaluation & Benchmarking Protocol **31**
12.1 50-Question Test Suite . . . 31
12.2 Scoring Criteria . . . 31
12.3 A/B Testing . . . 31

13 Appendix **32**
13.1 Diagram Index . . . 32
13.1.1 Presentation Diagrams . . . 32
13.1.2 Technical Diagrams . . . 32
13.2 Environment . . . 32

Chapter 1

Executive Summary

1.1 Project Overview

This project implements an automated preprocessing pipeline that transforms raw GitHub data from STMicroelectronics' public STM32Cube repositories into structured, RAG-ready (Retrieval-Augmented Generation) knowledge base content. The system feeds the **ST GitHub Analyzer** AI persona, enabling intelligent technical support for STM32 embedded systems engineers.

1.2 Key Achievements

- **5 STM32 series** fully processed: H7, F4, H5, U5, WL
- **40+ repositories** ingested (main repos + HAL/CMSIS/BSP drivers)
- **4 delivery artifact types**: Issues, Files, Resolver Cases, Diagnostic Cards
- **Automated KB upload** via ST AI Bridge API (KB #793)
- **Alfred Vision AI** integration for image-to-text enrichment
- **50-question test suite** for quality benchmarking

1.3 System Context

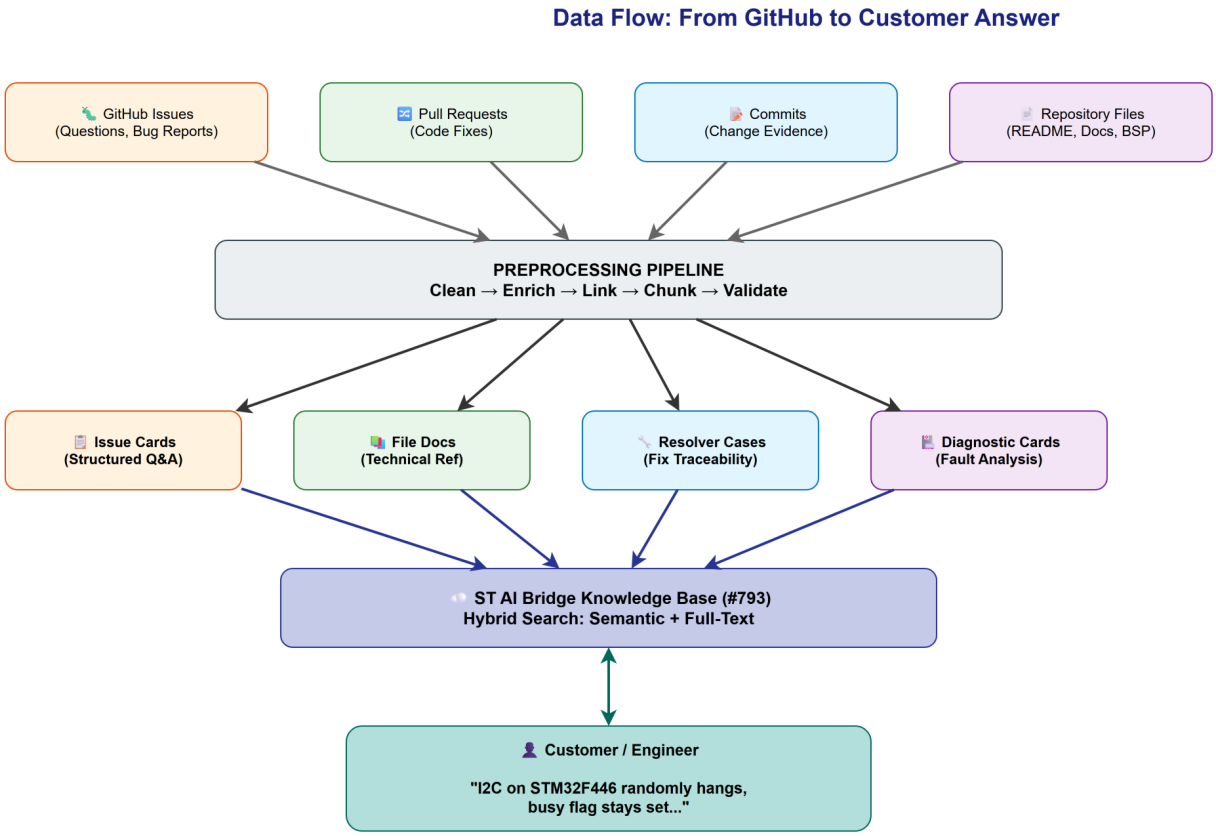


Figure 1.1: End-to-end data flow: from GitHub sources to chatbot answers

Chapter 2

Architecture

2.1 Pipeline Stages

The pipeline consists of 7 sequential stages, each reading from the previous stage's output:

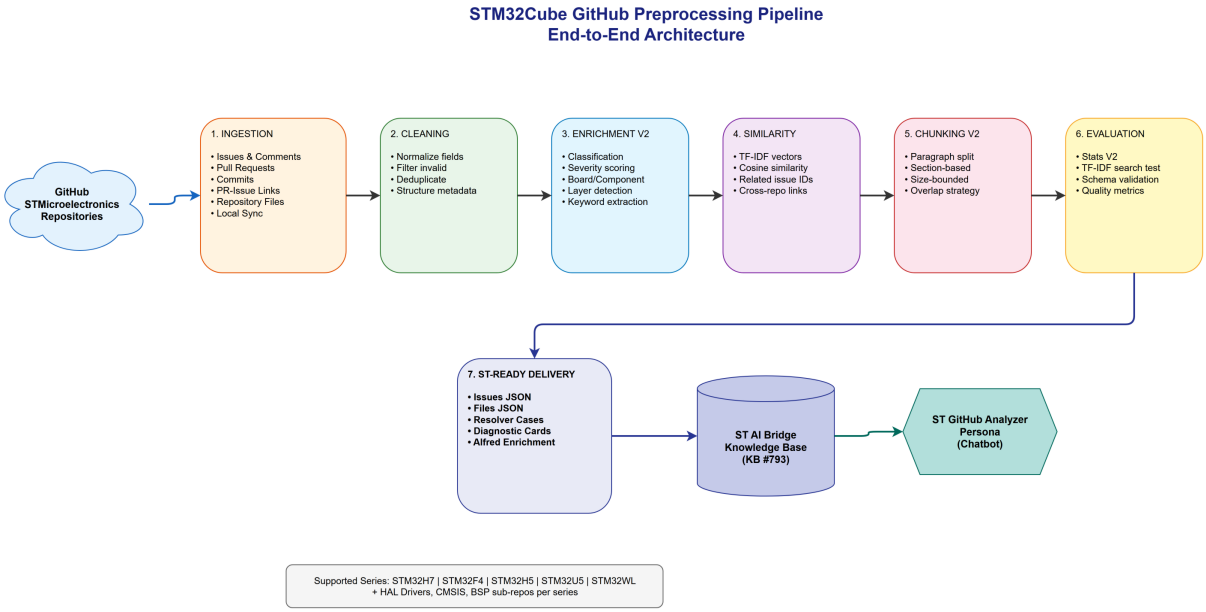


Figure 2.1: High-level pipeline architecture: 7 sequential stages

1. **Ingestion** — Fetch raw data from GitHub API and local clones
2. **Cleaning** — Normalize, deduplicate, structure metadata
3. **Enrichment V2** — Classify issues/files with heuristic NLP
4. **Similarity** — Compute TF-IDF cross-issue similarity
5. **Chunking V2** — Split documents for optimal retrieval
6. **Evaluation** — Stats, TF-IDF search tests, schema validation
7. **Delivery** — Export ST-ready JSON for KB upload

2.2 Repository Structure

```
PFE_Chatbot_STM32Cube/
|-- pipeline/                    # Core pipeline stages
|   |-- ingestion/               # GitHub API fetchers
|   |-- cleaning/                # Data normalization
|   |-- enrichment/              # NLP classification
|   |-- similarity/              # TF-IDF similarity
|   |-- chunking/                # Document splitting
|   |-- evaluation/              # Quality testing
|   |-- delivery/                # ST-ready export
|   +-- run_full_workflow.py     # Orchestrator
|-- pipeline_Automation/         # Upload & batch scripts
|-- shared/
|   |-- config/                  # Per-series config files
|   |-- schemas/                 # JSON Schema definitions
|   +-- utils/                   # Path helpers, utilities
|-- datasets/
|   |-- 01_raw/                  # Raw ingested data
|   +-- 07_delivery/st_ready/    # Final delivery outputs
|       +-- by_series/           # Organized per MCU series
|-- GitHub_repos/                # Local cloned repositories
|-- docs/                        # Documentation & diagrams
+-- data/                        # Historical/intermediate
```

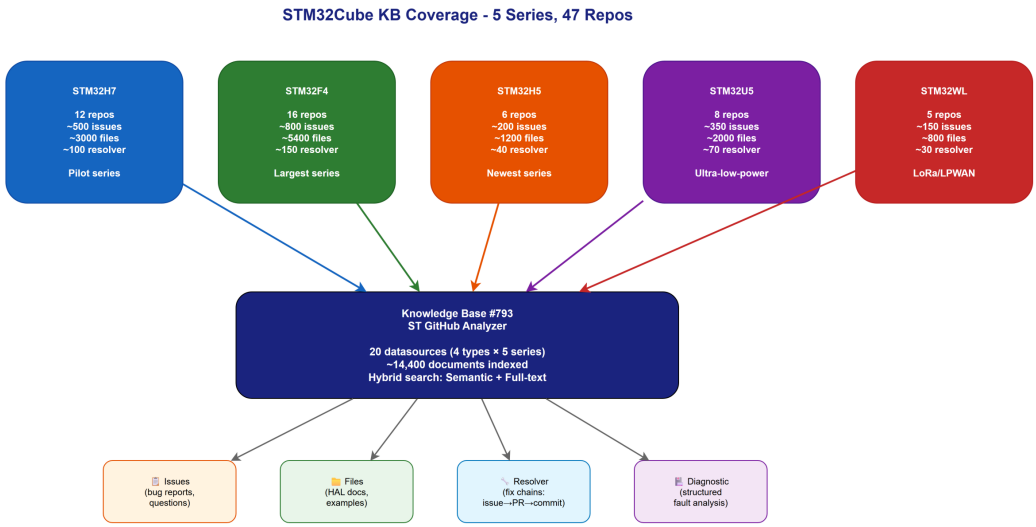


Figure 2.2: Multi-series coverage: 5 MCU families, 40+ repositories

2.3 Configuration System

Each MCU series has a dedicated configuration file:

- `shared/config/config.json` — Default (H7, 12 repos)

- `shared/config/config_f4.json` — STM32F4 (16 repos)
- `shared/config/config_h5.json` — STM32H5 (6 repos)
- `shared/config/config_u5.json` — STM32U5 (8 repos)
- `shared/config/config_wl.json` — STM32WL (5 repos)

Chapter 3

Pipeline Stage Details

3.1 Stage 1: Ingestion

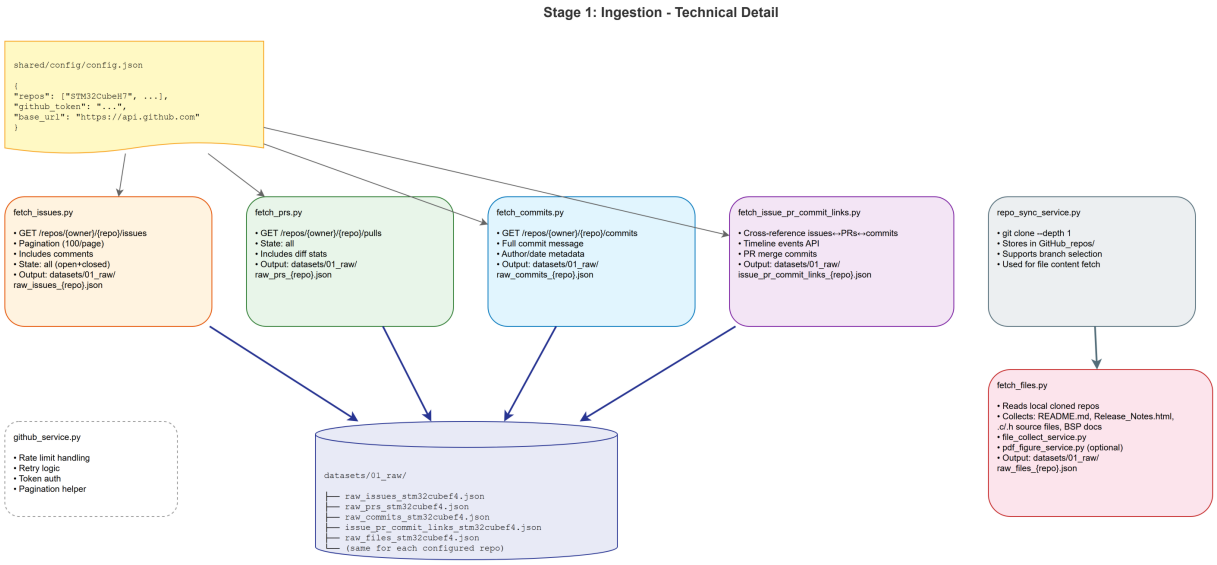


Figure 3.1: Ingestion stage: scripts, API calls, and output artifacts

3.1.1 Scripts

Script	Function
`fetch_issues.py`	Fetch all issues (open+closed) with comments
`fetch_prs.py`	Fetch pull requests with diff stats
`fetch_commits.py`	Fetch commits with messages
`fetch_issue_pr_commit_links.py`	Cross-reference issues↔PRs↔commits
`repo_sync_service.py`	Git clone/pull repositories locally
`fetch_files.py`	Extract file content from local clones
`github_service.py`	Shared: auth, rate-limiting, pagination

3.1.2 Output

All raw data is stored in `datasets/01_raw/` as JSON files named `raw_{type}_{repo}.json`.

3.2 Stage 2: Cleaning

- `clean_issues.py` — Normalize issue fields, filter spam, deduplicate
- `clean_files.py` — Normalize file paths, filter binary/irrelevant files

Output: `data/clean_issues_{repo}.json`, `data/clean_files_{repo}.json`

3.3 Stage 3: Enrichment V2

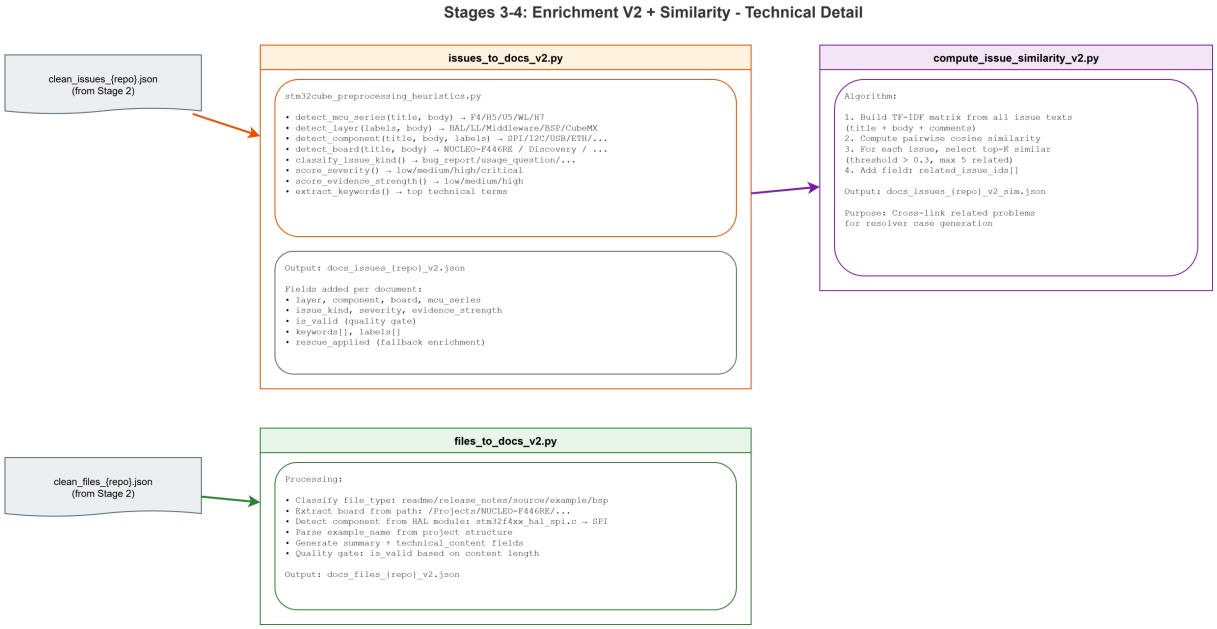


Figure 3.2: Enrichment & similarity stage: heuristic NLP classification and TF-IDF cross-linking

3.3.1 Issues Enrichment (`issues_to_docs_v2.py`)

Uses `stm32cube_preprocessing_heuristics.py` to classify each issue:

Field	Type	Values
`mcu_series`	string	F4, H5, H7, U5, WL
`layer`	string	HAL, LL, Middleware, BSP, CubeMX, Other
`component`	string	SPI, I2C, USB, ETH, DMA, ADC, ...

`board`	string	NUCLEO-F446RE, Discovery, ...
`issue_kind`	string	bug_report, usage_question, config_help, other
`severity`	string	low, medium, high, critical
`evidence_strength`	string	low, medium, high
`is_valid`	boolean	Quality gate for delivery
`keywords`	string[]	Top technical terms
`rescue_applied`	boolean	Fallback enrichment used

3.3.2 Files Enrichment (`files_to_docs_v2.py`)

Classifies repository files by type, board, component, and extracts structured content.

3.4 Stage 4: Similarity

`compute_issue_similarity_v2.py`:

1. Builds TF-IDF matrix from issue text corpus
2. Computes pairwise cosine similarity
3. Attaches `related_issue_ids[]` (top-5, threshold > 0.3)

Output: `docs_issues_{repo}_v2_sim.json`

3.5 Stage 5: Chunking V2

- `docs_to_chunks_issues_v2.py` — Paragraph-based chunking with overlap
- `docs_to_chunks_files_v2.py` — Section-based chunking for file docs

Strategy: bounded chunks (max 512 tokens) with 50-token overlap for context continuity.

3.6 Stage 6: Evaluation

Script	Purpose
`test_stats_issues_v2.py`	Distribution analysis: severity, layer, component
`test_stats_files_v2.py`	File type distribution, board coverage
`test_tfidf_search_issues_v2.py`	TF-IDF retrieval precision testing
`test_tfidf_search_files_v2.py`	File retrieval precision testing
`validate_schemas.py`	JSON Schema validation gate

3.7 Stage 7: Delivery

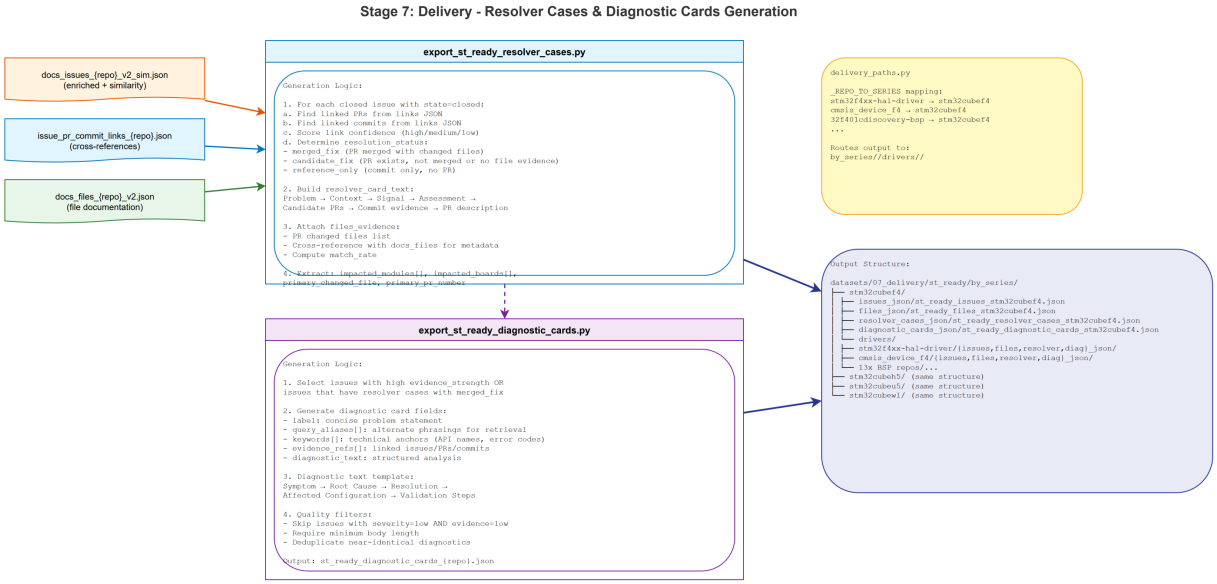


Figure 3.3: Delivery stage: resolver cases and diagnostic card generation

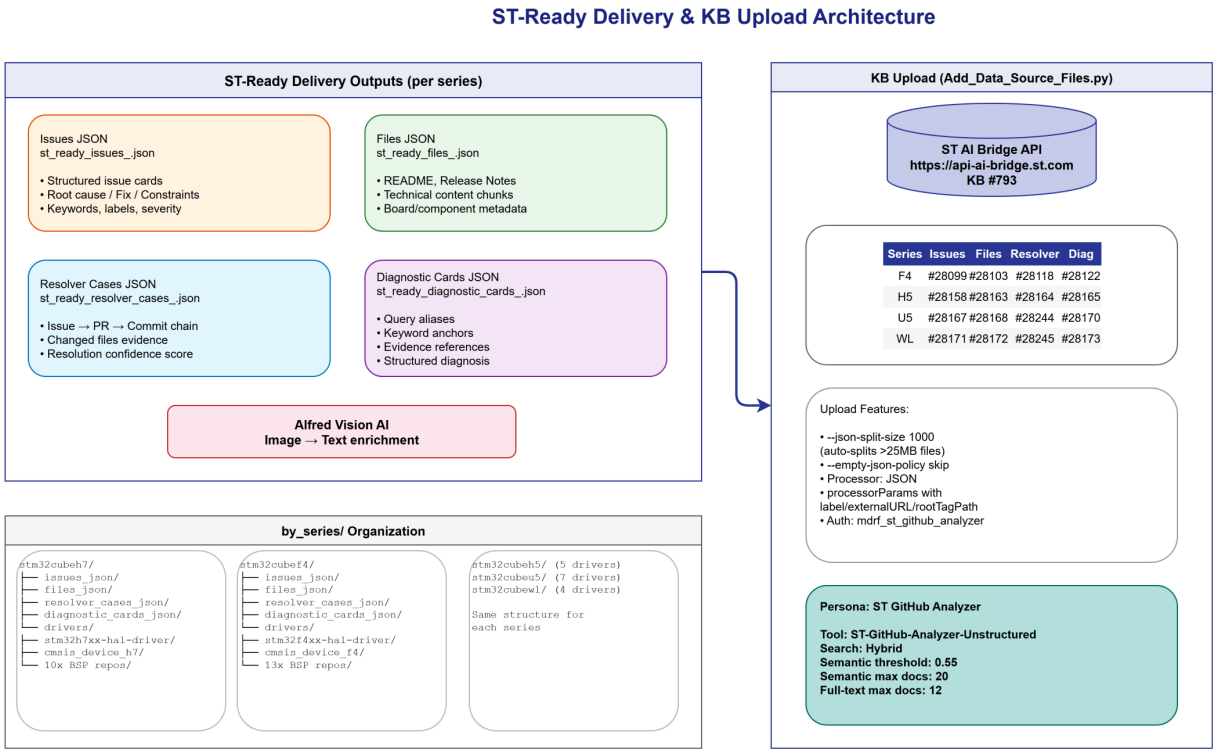


Figure 3.4: Delivery artifacts and KB upload workflow

3.7.1 Export Scripts

- `export_st_ready_issues.py` — Structured issue cards
- `export_st_ready_files.py` — Technical file documentation
- `export_st_ready_resolver_cases.py` — Issue→PR→Commit fix chains
- `export_st_ready_diagnostic_cards.py` — Structured fault analysis

3.7.2 Delivery Path Routing

`delivery_paths.py` contains a `_REPO_TO_SERIES` mapping that routes sub-repo outputs to the correct parent series folder:

```
by_series/stm32cubef4/drivers/stm32f4xx-hal-driver/issues_json/
by_series/stm32cubef4/drivers/cmsis_device_f4/files_json/
```

Chapter 4

KB Upload & Deployment

4.1 ST AI Bridge API

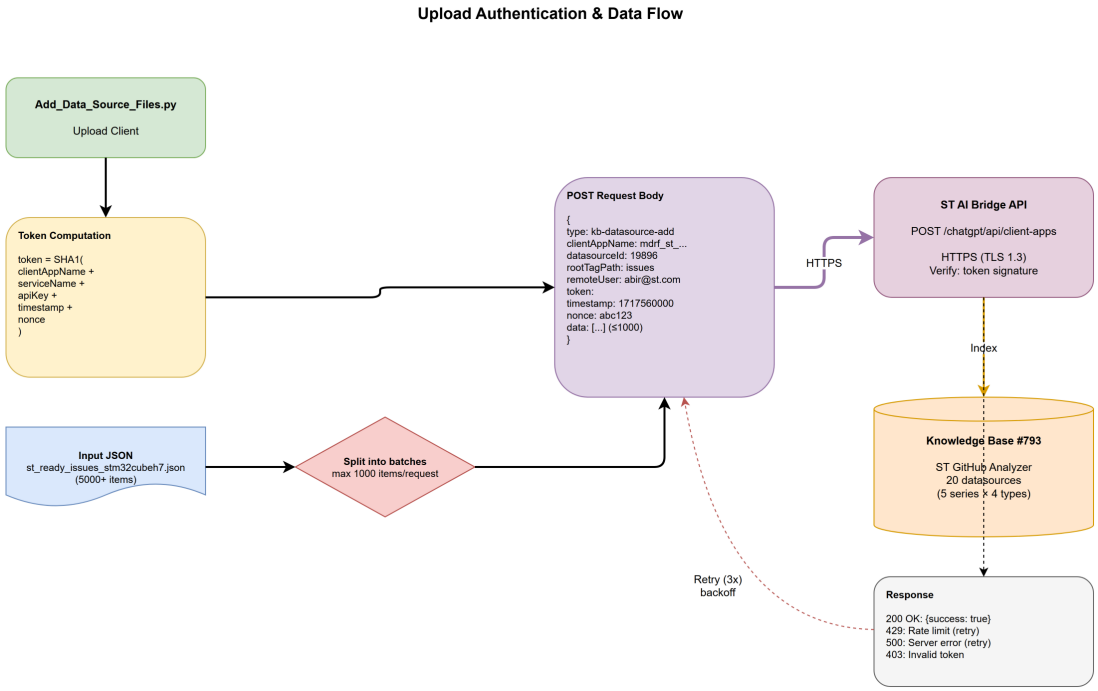


Figure 4.1: Upload authentication flow: API key, client app, and datasource creation

- **Endpoint**: `https://api-ai-bridge.st.com/chatgpt/api/client-apps`
- **KB ID**: 793 (ST GitHub Analyzer)
- **Auth**: clientAppName=`mdrf_st_github_analyzer`
- **Upload tool**: `Add_Data_Source_Files.py`

4.2 Datasource Registry

Series	Issues	Files	Resolver	Diagnostic
F4	#28099	#28103	#28118	#28122
H5	#28158	#28163	#28164	#28165
U5	#28167	#28168	#28244	#28170
WL	#28171	#28172	#28245	#28173

4.3 Upload Parameters

Each datasource type uses specific processor parameters:

```
// Issues
{"label": "{{repo}} issue #{{issue_number}} - {{issue_title}}",
 "externalURL": "https://github.com/STMicroelectronics/{{repo}}/
    issues/{{issue_number}}",
 "rootTagPath": "issues"}

// Files
{"label": "{{repo}} {{file_type}} - {{path}}",
 "externalURL": "https://github.com/STMicroelectronics/{{repo}}/{{
    root_dir}}/{{root_ref}}/{{path_url}}",
 "rootTagPath": "files"}

// Resolver Cases
{"label": "{{label}}",
 "externalURL": "https://github.com/STMicroelectronics/{{repo}}/
    issues/{{issue_number}}",
 "rootTagPath": "resolver_cases"}

// Diagnostic Cards
{"label": "{{label}}",
 "externalURL": "https://github.com/STMicroelectronics/{{repo}}/
    issues/{{evidence_refs[0].id}}",
 "rootTagPath": "diagnostic_cards"}
```

4.4 Large File Handling

Files exceeding Azure Blob limits (>10MB) are automatically split:

- `-json-split-size 1000` splits arrays into 1000-doc parts
- Example: `st_ready_files_stm32cubef4.json` (25.5MB, 5397 items) → 6 parts

Chapter 5

Alfred Vision AI Integration

5.1 Purpose

Many GitHub issues contain screenshots (error dialogs, logic analyzer traces, CubeMX configurations). These images are opaque to text-based RAG retrieval. Alfred converts images to text descriptions.

5.2 Integration

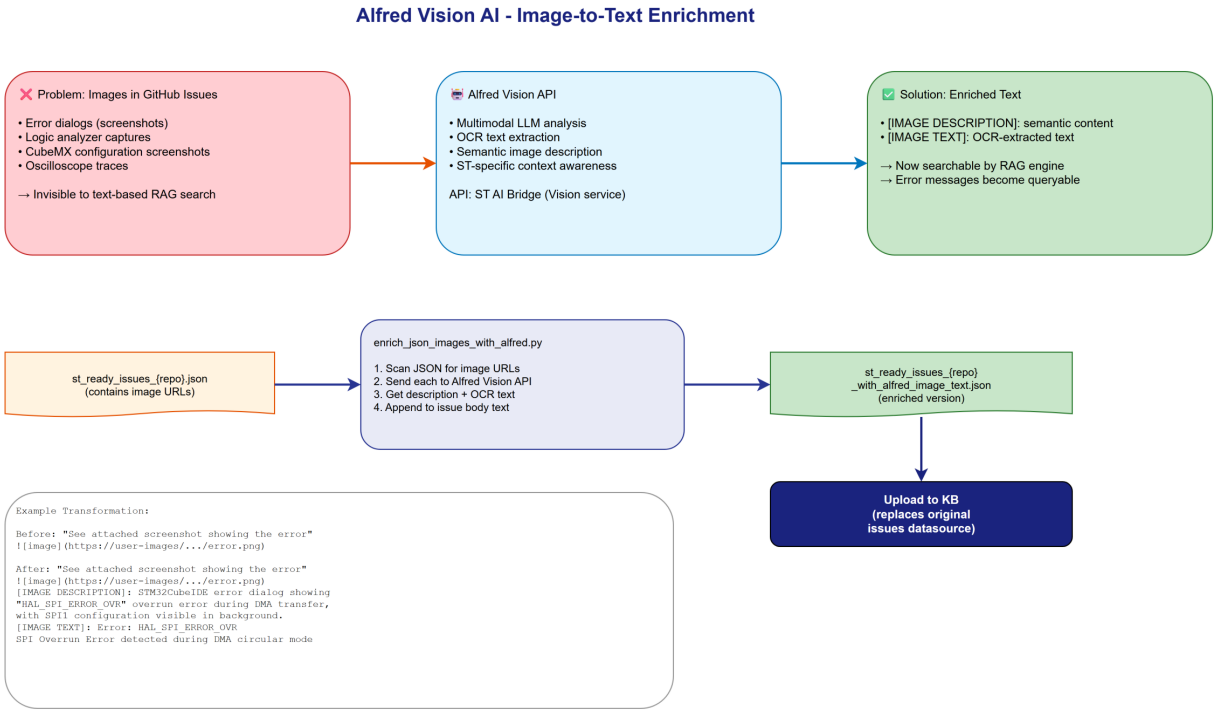


Figure 5.1: Alfred Vision AI integration: image URL extraction, API call, text injection

```
python pipeline_Automation/enrich_json_images_with_alfred.py \
  --input "datasets/.../st_ready_issues_stm32cubef4.json" \
```

```
--persona "st_copilot" --strip-analyses
```

Output: `st_ready_issues_{repo}_with_alfred_image_text.json`
Each image URL in issue text gets appended fields:

- `[IMAGE DESCRIPTION]` — Semantic description of the image content
- `[IMAGE TEXT]` — OCR-extracted text from the image

Chapter 6

Persona Configuration

6.1 Recommended Settings

Parameter	Value
Tool Name	ST-GitHub-Analyzer-Unstructured
Search Type	Hybrid (Semantic + Full-text)
Semantic Threshold	0.55
Semantic Max Doc Count	20
Full-text Max Doc Count	12

6.2 System Prompt

The persona system prompt follows a 4-case answer strategy:

1. Direct KB match → cite source
2. Partial match → diagnosis + candidate workaround + validation steps
3. General configuration question → direct technical explanation
4. No match → actionable next step (never only "I don't know")

Chapter 7

Automation & Operations

7.1 Full Workflow Execution

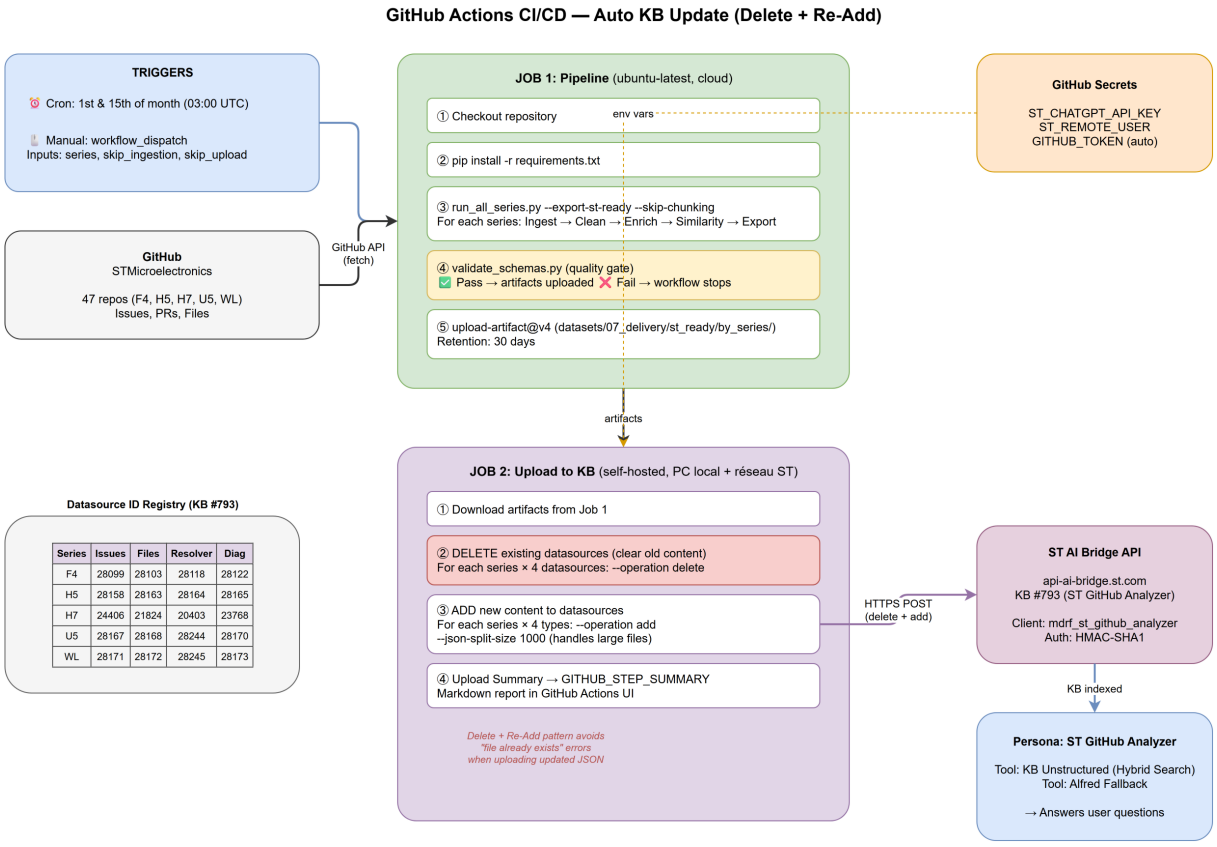


Figure 7.1: CI/CD automation: GitHub Actions workflow with delete+add upload pattern

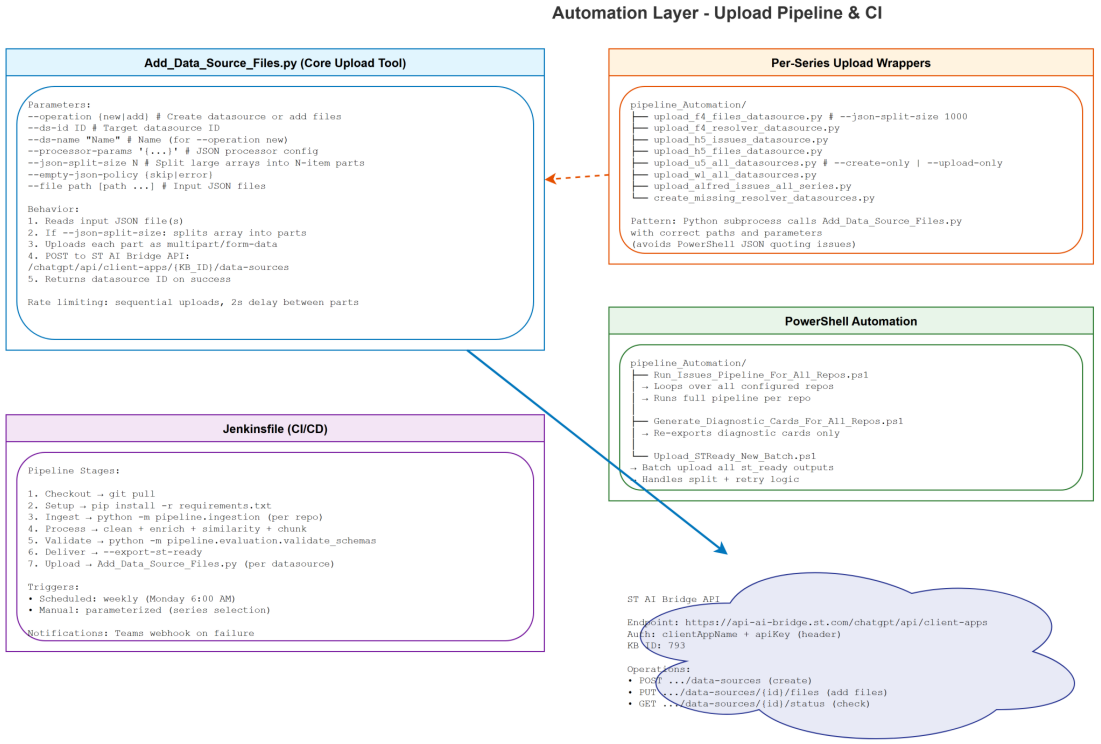


Figure 7.2: Automation architecture: scheduled triggers, validation gates, and upload

```
# Standard run
python -m pipeline.run_full_workflow

# With delivery export
python -m pipeline.run_full_workflow --export-st-ready

# Skip ingestion (reprocess from existing raw data)
python -m pipeline.run_full_workflow --skip-ingestion --continue-on
    -error
```

7.2 Multi-Series Automation

```
# Run all series sequentially
python -m pipeline_Automation.run_all_series

# Re-export delivery only (no re-ingestion)
python -m pipeline_Automation.reexport_delivery --series f4 --
    continue-on-error
```

7.3 Upload Scripts

Per-series upload scripts handle datasource creation and file upload:

- `upload_f4_files_datasource.py`, `upload_f4_resolver_datasource.py`, etc.
- `upload_u5_all_datasources.py` — Creates + uploads all 4 types
- `upload_alfred_issues_all_series.py` — Uploads Alfred-enriched versions

7.4 Fully Automated KB Update (End-to-End)

The script `pipeline_Automation/workflow/Run_Full_Auto_Update_KB.ps1` chains **all stages** in a single command:

```
# Full auto-update: ingest + preprocess + export + validate +
   upload
.\pipeline_Automation\workflow\Run_Full_Auto_Update_KB.ps1 `
    -RemoteUser "abir.gharbi@st.com" `
    -Series f4,h5,h7,u5,wl

# Re-process only (skip GitHub fetch, keep raw data)
.\pipeline_Automation\workflow\Run_Full_Auto_Update_KB.ps1 `
    -RemoteUser "abir.gharbi@st.com" `
    -SkipIngestion

# Pipeline only, no KB upload (dry-run)
.\pipeline_Automation\workflow\Run_Full_Auto_Update_KB.ps1 `
    -SkipUpload -Series h7
```

7.4.1 What the Script Does

1. Runs `run_all_series.py` with `-export-st-ready` for each selected MCU series
2. Validates all outputs with `validate_schemas`
3. **Blocks upload if validation fails** (safety gate)
4. Uploads each artifact type to its registered datasource ID in KB #793
5. Writes a timestamped log to `logs/auto_update_kb_*.log`

7.4.2 Scheduling for Always-Fresh KB

Note: Local chunking is skipped (`-skip-chunking`) because the ST ChatGPT platform handles document chunking internally at upload time.

Method	Setup
GitHub Actions (recommended)	Workflow file `.github/workflows/auto_update_kb.yml` runs automatically every 2 weeks (1st and 15th of each month). Fully cloud-based — no local machine needed.

Jenkins (CI/CD)	Use the existing `Jenkinsfile` with a cron trigger: `H 2 1,15 * *` (bi-weekly). Add the Upload stage.
Local (fallback)	Windows Task Scheduler running `Run_Full_Auto_Update_KB.ps1` every 2 weeks. Requires machine to be powered on.

7.4.3 GitHub Actions Cloud Automation (Recommended)

The workflow `.github/workflows/auto_update_kb.yml` provides fully automated cloud execution:

- **Trigger**: Bi-weekly cron (`0 3 1,15 * *`) + manual dispatch
- **Steps**: Ingest → Preprocess (no chunking) → Export → Validate → Upload → Archive
- **Secrets required**: `ST_CHATGPT_API_KEY`, `ST_REMOTE_USER`, `GITHUB_TOKEN`
- **No local machine needed**: Runs entirely on GitHub-hosted runners
- **Manual override**: Can be triggered on-demand via "Run workflow" button with custom series selection

To set up:

1. Go to repository Settings → Secrets and variables → Actions
2. Add secret `ST_CHATGPT_API_KEY` with your ST AI Bridge API key
3. Add secret `ST_REMOTE_USER` with your email (e.g. `abir.gharbi@st.com`)
4. Push the `.github/workflows/auto_update_kb.yml` file to the main branch
5. The workflow will trigger automatically on the 1st and 15th of each month

7.4.4 Recommended Frequency

- **Weekly**: Catches new issues and community activity
- **On firmware release**: Run immediately after a new CubeF4/H7/etc. version is tagged
- **On-demand**: Before demos, evaluations, or after pipeline code changes

7.4.5 Safety Guarantees

- Schema validation gate prevents corrupted data from reaching the KB
- `kb-datasource-add` replaces existing content (not append), so no duplicates
- Summary JSON files are generated for traceability (count verification)
- Logs capture every step for post-mortem analysis
- `-ContinueOnError` allows partial series to complete even if one fails

Chapter 8

Quality Assurance

8.1 Test Suite

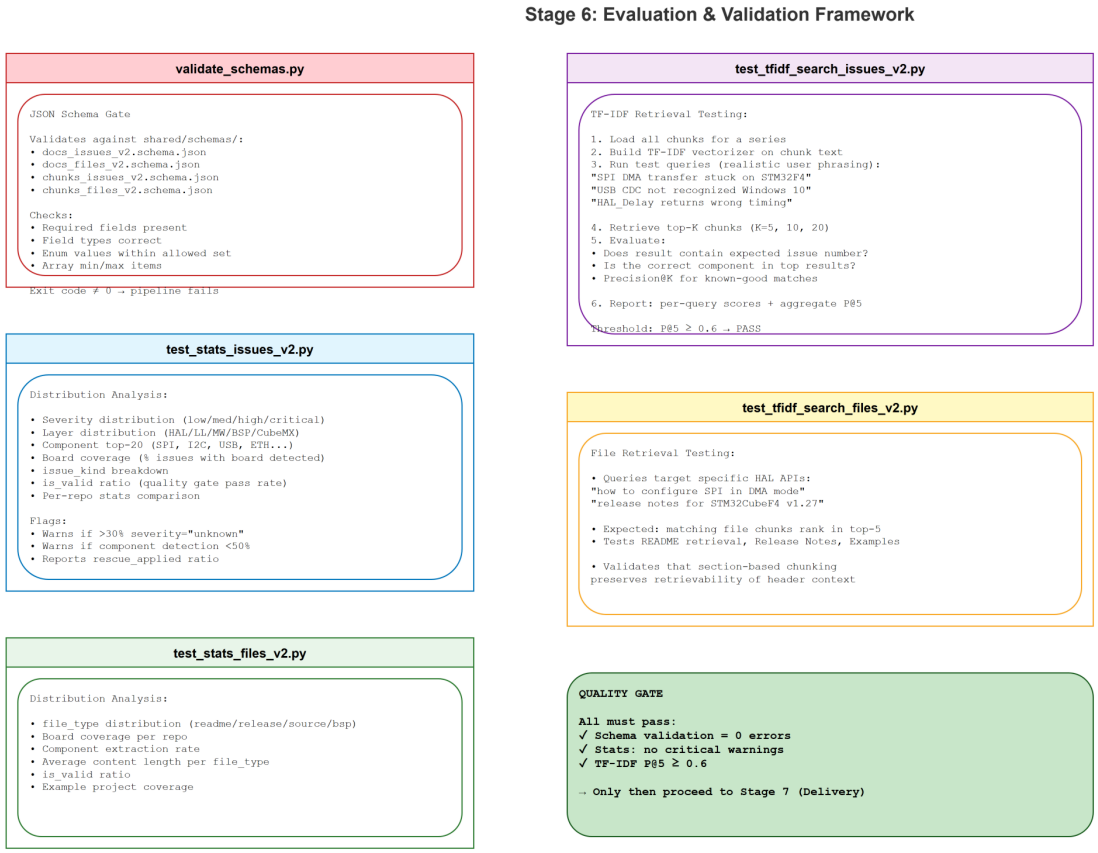


Figure 8.1: Evaluation and validation pipeline: stats, TF-IDF tests, schema gates

50-question test suite (`all_series_test_suite_50q_v2.csv`):

- 13 questions STM32F4
- 12 questions STM32H5
- 13 questions STM32U5

- 12 questions STM32WL

Questions are written in realistic customer voice with symptoms, board context, and expected technical anchors.

8.2 Schema Validation

JSON schemas under `shared/schemas/` enforce structural contracts:

- `docs_issues_v2.schema.json`
- `docs_files_v2.schema.json`
- `chunks_issues_v2.schema.json`
- `chunks_files_v2.schema.json`

Run: `python -m pipeline.evaluation.validate_schemas`

Chapter 9

ST ChatGPT Platform — Usage Guide

9.1 Platform Overview

The ST ChatGPT platform (`https://stchatgpt.st.com`) is ST's internal deployment of Azure OpenAI, extended with:

- **Personas** — Custom AI agents with dedicated knowledge bases and system prompts
- **Knowledge Bases (KB)** — Uploaded document collections enabling RAG retrieval
- **Tools** — Search tools (Hybrid, Semantic, Full-text) connecting the LLM to KB content
- **AI Bridge API** — REST API for programmatic KB management and datasource uploads

9.2 How the Workflow Operates End-to-End

1. **Data Ingestion**: GitHub issues, PRs, commits, and file content are fetched via GitHub API and local repository clones.
2. **Preprocessing**: Raw data is cleaned, enriched with heuristic NLP classification (layer, component, severity, board), and cross-linked via TF-IDF similarity.
3. **Chunking**: Documents are split into retrieval-optimal chunks (max 512 tokens, 50-token overlap).
4. **ST-Ready Export**: Four delivery artifact types are generated as JSON (issues, files, resolver cases, diagnostic cards).
5. **KB Upload**: JSON artifacts are uploaded to KB #793 via the AI Bridge API, creating indexed datasources.
6. **Persona Answers**: When a user asks a question, the persona searches the KB, retrieves relevant chunks, and generates a structured answer citing sources.

```
User Question
  -> Persona System Prompt (answer strategy)
    -> Tool: ST-GitHub-Analyzer-Unstructured (Hybrid Search)
      -> KB #793 (Semantic + Full-text index)
        -> Retrieved chunks (issues, files, resolver, diagnostic)
          -> LLM generates structured answer with citations
```

9.3 Knowledge Base Content Structure

KB #793 contains 4 datasource types per MCU series (F4, H5, H7, U5, WL):

Datasource Type	File Pattern	Content
Issues	`st_ready_issues_*.json`	Structured issue cards with metadata, symptoms, root cause, fix
Files	`st_ready_files_*.json`	Technical documentation from HAL drivers, BSP, CMSIS, examples
Resolver Cases	`st_ready_resolver_*.json`	Issue→PR→Commit fix chains with traceability
Diagnostic Cards	`st_ready_diagnostic_*.json`	Structured fault analysis with hypotheses, checks, workarounds

9.3.1 File Organization in KB

Each series groups its main repo and sub-repositories (drivers, BSP, CMSIS):

```
by_series/
|-- stm32cubef4/
|   |-- issues_json/          # Main repo issues
|   |-- files_json/           # Main repo files
|   |-- resolver_cases_json/  # Fix chains
|   |-- diagnostic_cards_json/# Diagnostic analysis
|   +-- drivers/              # Sub-repos (HAL, BSP, CMSIS)
|       |-- stm32f4xx-hal-driver/
|       |-- cmsis_device_f4/
|       +-- stm32f4xx-nucleo-bsp/
|-- stm32cubeh7/
|-- stm32cubeh5/
|-- stm32cubeu5/
+-- stm32cubewl/
```

9.3.2 What Each Datasource Enables

- **Issues**: Answers questions about known bugs, workarounds, and community-reported problems

- **Files**: Answers questions about HAL/LL API usage, driver configuration, BSP initialization, release notes
- **Resolver Cases**: Enables tracing from symptom to exact commit/PR fix
- **Diagnostic Cards**: Provides structured troubleshooting flow (facts → hypotheses → checks → workaround)

Chapter 10

Persona Configuration — Best Practices

10.1 Recommended Persona Settings

Parameter	Recommended	Rationale
Tool Name	ST-GitHub-Analyzer-Unstructured	Hybrid search tool
Search Type	Hybrid	Combines semantic understanding + exact keyword matching
Semantic Threshold	0.55	Below 0.5: too much noise; above 0.65: misses partial matches
Semantic Max Doc Count	20	Balances coverage with context window limits
Full-text Max Doc Count	12	Ensures keyword-heavy queries (API names, error codes) find exact matches
Temperature	0.2–0.3	Low = factual, deterministic answers
Max Tokens (response)	2000–3000	Enough for structured diagnosis; not wasteful

10.2 System Prompt Design

The persona system prompt must enforce:

1. **4-case answer strategy**:
    - Direct KB match → cite source with GitHub link
    - Partial match → diagnosis + candidate workaround + validation steps

- General configuration question → direct technical explanation
- No match → actionable next step (never only "I don't know")

2. **Mandatory clarification questions**: The persona must always ask follow-up questions (exact MCU part, board, firmware version, reproduction steps) before providing a definitive fix.

3. **Citation format**: Every answer must include the source GitHub URL and issue/file reference.

4. **Scope declaration**: The prompt must declare which MCU series are covered and redirect gracefully for unsupported series.

10.3 What Works Well (Proven Patterns)

- **API-name anchoring**: Questions mentioning exact HAL/LL function names (e.g., `HAL_SPI_Transmit_DMA`) retrieve with high precision because diagnostic cards include `query_aliases` and `api_family_tags`.
- **Board + component queries**: "NUCLEO-F446RE SPI timeout" retrieves accurately thanks to enriched metadata fields.
- **Symptom-based retrieval**: Diagnostic cards contain `symptom_summary` and `technical_patter` enabling natural-language symptom matching.
- **Version-specific questions**: `version_tags` enable retrieval for upgrade/regression scenarios.

10.4 Known Limitations

Limitation	Impact & Mitigation
Context window saturation	With 20 semantic + 12 full-text docs, complex queries may hit token limits. Mitigation: chunking keeps individual docs small ($\leq$512 tokens).
Image content opacity	Screenshots in issues are opaque to text search. Mitigation: Alfred Vision AI enrichment converts images to text descriptions.
Stale data	KB content reflects the last pipeline run. Mitigation: schedule periodic re-runs (weekly or after major Cube releases).
Cross-series confusion	Questions without explicit series may retrieve answers from wrong MCU family. Mitigation: system prompt enforces clarification questions.

Low-signal issues	Some GitHub issues contain only conversational noise ("thank you", "+1"). Mitigation: `is_valid` filter and `rescue_applied` heuristic.
Large file splits	Files >10MB require splitting into parts, which may fragment context. Mitigation: `-json-split-size 1000` keeps parts manageable.

10.5 Anti-Patterns to Avoid

- **DO NOT** set semantic threshold below 0.4 — retrieves too much irrelevant content
- **DO NOT** disable full-text search — API names and error codes need exact matching
- **DO NOT** upload raw GitHub JSON without preprocessing — the persona cannot parse unstructured API responses
- **DO NOT** skip diagnostic cards — they are the primary structured troubleshooting source
- **DO NOT** use temperature > 0.5 for technical answers — causes hallucination of non-existent fixes
- **DO NOT** merge all series into a single datasource — separate datasources per series improve precision and allow targeted updates

Chapter 11

KB Maintenance & Update Workflow

11.1 When to Re-run the Pipeline

- After a new STM32Cube firmware release (e.g., v1.29.0)
- After significant issue activity (20+ new issues since last run)
- After adding support for a new MCU series
- Before a major demo or evaluation cycle

11.2 Update Procedure

```
# 1. Re-run pipeline with fresh ingestion
python -m pipeline.run_full_workflow --export-st-ready

# 2. Validate outputs
python -m pipeline.evaluation.validate_schemas

# 3. Upload new batch to KB
python pipeline_Automation/upload/Add_Data_Source_Files.py \
  --input "datasets/07_delivery/st_ready/by_series/stm32cubef4/
     issues_json/*.json" \
  --datasource-id 28099 --json-split-size 1000

# 4. Verify in ST ChatGPT platform
# -> Open persona -> Test a known question -> Confirm source
   citations
```

11.3 Datasource Versioning Strategy

- Each upload **replaces** the previous datasource content (not append).
- Keep the previous JSON export as backup before uploading.

- Use `summary_*.json` files to track counts and verify no data loss.
- Monitor for regressions using the 50-question test suite after each upload.

11.4 Adding a New MCU Series

1. Create `shared/config/config_{series}.json` with the repository list
2. Run: `python -m pipeline.run_full_workflow -config config_{series}.json -export-st-ready`
3. Create 4 datasources in KB #793 (issues, files, resolver, diagnostic)
4. Upload the generated JSON files
5. Add 10–15 test questions covering the new series
6. Update the persona system prompt to declare the new series as supported

Chapter 12

Evaluation & Benchmarking Protocol

12.1 50-Question Test Suite

The evaluation protocol uses a curated question set with:

- Realistic customer voice (symptoms, not solutions)
- Board and version context included
- Expected technical anchors documented for each question
- Coverage across all 5 series and multiple issue types

12.2 Scoring Criteria

Each answer is scored on 4 dimensions:

Dimension	Weight	Criteria
Retrieval Relevance	30%	Did the persona find the right KB document?
Technical Accuracy	30%	Is the answer factually correct?
Actionability	25%	Can the user act on the response without further research?
Citation Quality	15%	Are sources cited with valid GitHub URLs?

12.3 A/B Testing

When testing configuration changes (threshold, doc count, prompt wording):

1. Run the 50-question suite on configuration A (baseline)
2. Apply the change and run on configuration B
3. Compare scores per dimension
4. Only deploy B if it improves retrieval relevance without degrading accuracy

Chapter 13

Appendix

13.1 Diagram Index

13.1.1 Presentation Diagrams

1. `01_pipeline_overview.drawio` — High-level 7-stage pipeline
2. `02_delivery_kb_upload.drawio` — Delivery artifacts + KB upload
3. `03_data_flow_source_to_answer.drawio` — End-to-end data flow

13.1.2 Technical Diagrams

1. `01_ingestion_detail.drawio` — Ingestion scripts and outputs
2. `02_enrichment_similarity_detail.drawio` — Heuristic classification
3. `03_delivery_resolver_diagnostic.drawio` — Resolver/diagnostic generation

13.2 Environment

- Python 3.14+
- Virtual environment: `.venv/`
- Key dependencies: requests, scikit-learn (TF-IDF), openpyxl
- OS: Windows 11 (OneDrive workspace)
- IDE: VS Code with Copilot